7강. 객체지향 프로그래밍

- 객체지향 프로그래밍이란?
- 객체란?
- 객체의 상호작용
- 객체 간의 관계
- 객체 지향 프로그램 특징
- 객체와 클래스

7-1. 객체지향 프로그래밍이란?

현실세계에서 어떤 제품을 만들 때는 부품을 먼저 만들고, 이 부품들을 하나씩 조립해서 완성품을 만듭니다. 소프트웨어를 개발할 때에도 부품에 해당하는 객체들을 먼저 만들고, 이 객체들을 하나씩 조립해서 완성된 프로그램을 만드는 기법을 객체 지향 프로그래밍(Object Oriented Programming: OOP) 이라고 합니다.

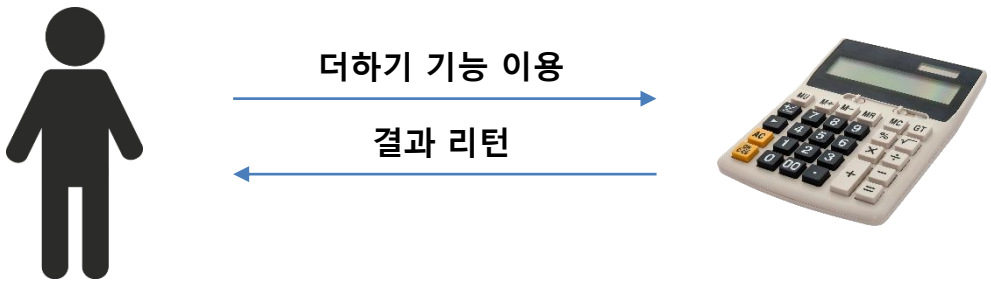
7-2. 객체란?

- 객체(Object)란 물리적으로 존재하거나 개념적인 것 중에서 다른 것과 식별 가능한 것을 말합니다. 예를 들어, 물리적으로 존재하는 자동차, 자전거, 책, 사람은 물론이고 개념적인 학과, 강의, 주문 등도 모두 객체가 될 수 있습니다.
- 객체는 속성과 동작으로 구성됩니다. 예를 들어 사람을 보면 이름, 나이 등의 속성과 웃다, 걷다 등의 동작이 있고, 자동차는 색상, 모델명 등의 속성과 달린다, 멈춘다 등의 동작이 있습니다. Java에서는 이러한 속성과 동작을 각각 필드(field)와 메서드(method)라고 부릅니다.

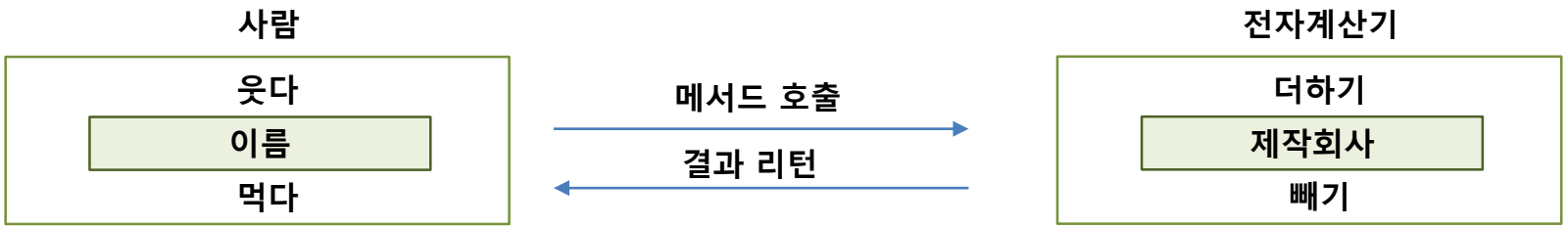
- 현실 세계의 객체를 소프트웨어 객체로 설계하는 것을 객체 모델링(object modeling)이라도 합니다. 객체 모델링은 현실 세계 객체의 대표 속성과 동작을 추려내어 소프트웨어 객체의 필드와 메서드로 정의하는 과정이라고 볼 수 있습니다.

7-3. 객체의 상호작용

- 현실 세계에서 일어나는 모든 현상은 객체와 객체 간의 상호작용으로 이루어져 있습니다. 예를 들어 사람은 전자계산기의 기능을 이용하고, 전자계산기는 결과를 사람에게 리턴하는 상호작용을 합니다.

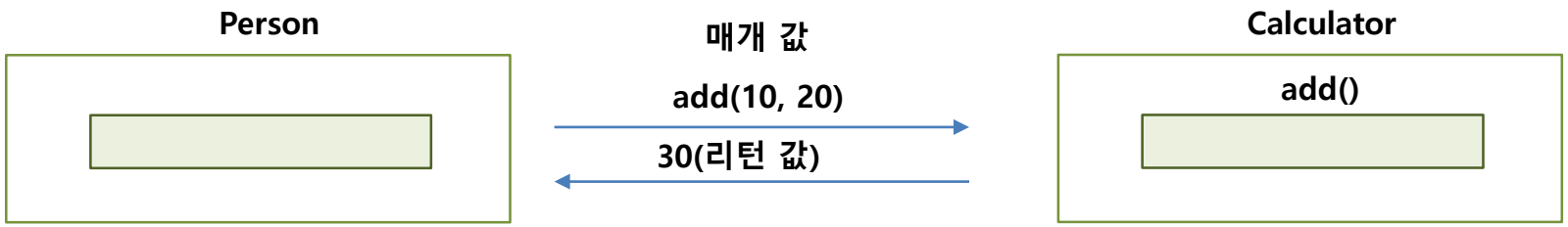


- 객체 지향 프로그램에서도 객체들은 다른 객체와 서로 상호작용하면서 동작한다. 객체들 사이의 상호작용 수단은 메서드 입니다. 객체가 다른 객체의 기능을 이용할 때 이 메서드를 호출합니다.



7-3. 객체의 상호작용

➢ 메서드 호출을 통해 객체들은 데이터를 서로 주고 받는다. 메서드 이름과 함께 전달하고자 하는 데이터를 괄호() 안에 기술하는데, 이러한 데이터를 매개 값이라고 한다. 이 매개 값은 메서드가 실행할 때 필요한 값이다. 리턴 값은 메서드의 실행의 결과이며, 호출한 곳으로 돌려주는 값이다.



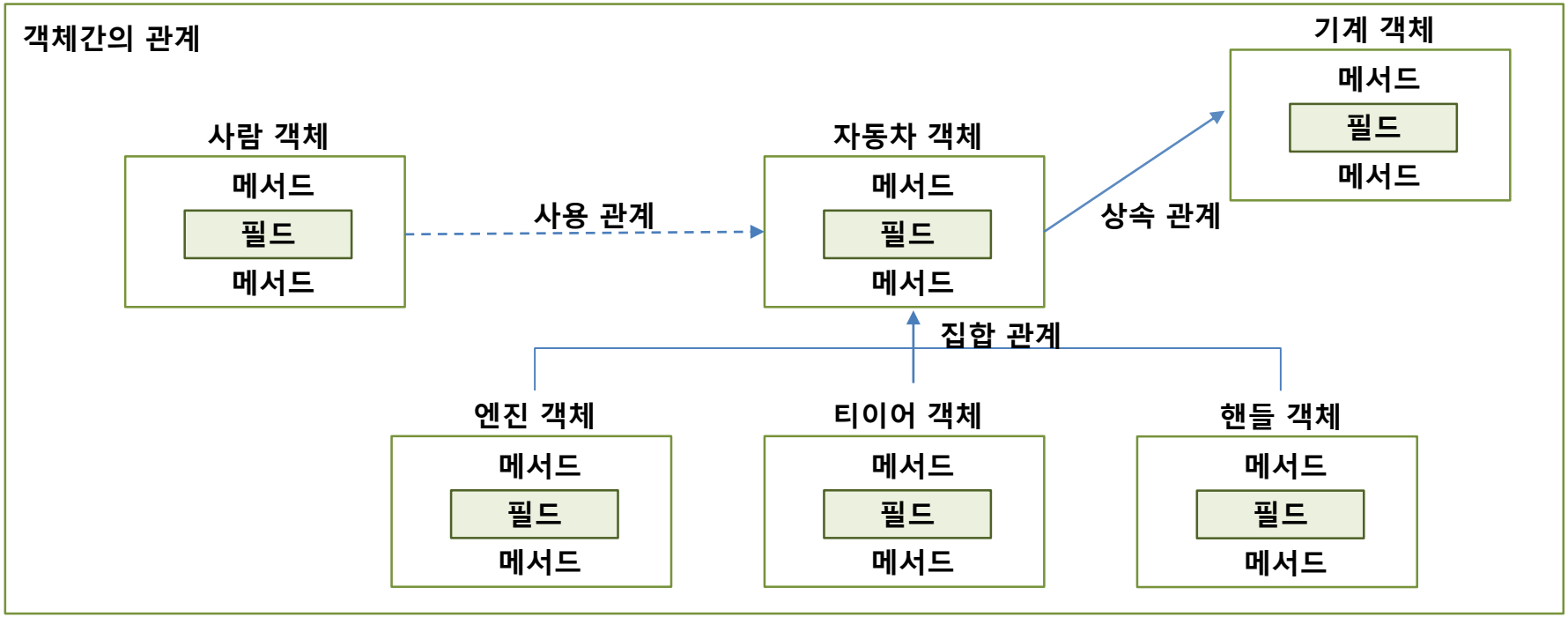
➢ 메서드의 리턴 값은 아래와 같이 호출한 곳에서 변수로 대입 받아 사용한다.

```
int result = add(10, 20);
```

리턴한 값을 int 변수에 저장

7-4. 객체간의 관계

➢ 객체는 단독으로 존재할 수 있지만 대부분 다른 객체와 관계를 맺고 있다. 관계의 종류에는 집합 관계, 사용 관계, 상속 관계가 있다.



집합 관계

완성품과 부품의 관계를 말한다. 예를 들어 자동차는 엔진, 타이어, 핸들 등으로 구성되므로 자동차와 부품들은 집합 관계라고 볼 수 있다.

7-4. 객체간의 관계

사용 관계

다른 객체의 필드를 읽고 변경하거나 메서드를 호출하는 관계를 말한다. 예를 들어 사람이 자동차에게 달린다, 멈춘다 등의 메서드를 호출하면 사람과 자동차는 사용 관계라고 불 수 있다.

상속 관계

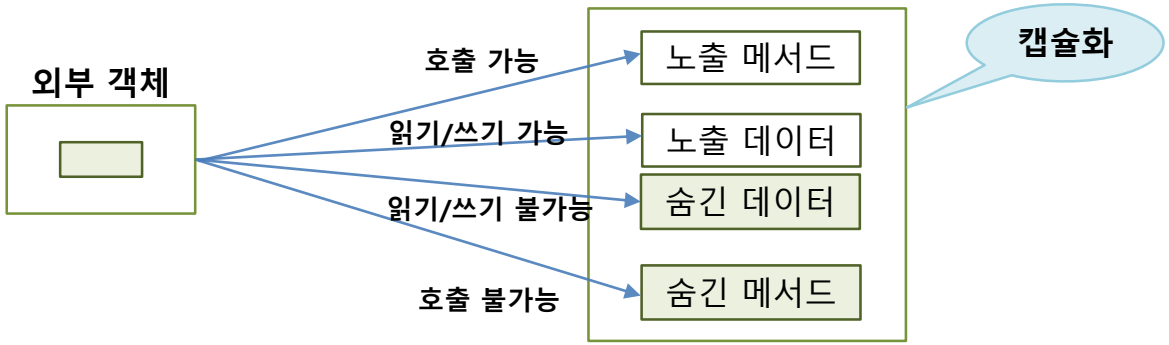
부모와 자식 관계를 말한다. 자동차가 기계의 특징(필드, 메서드)을 물려 받는다면 기계(부모)와 자동차(자식)는 상속 관계에 있다고 볼 수 있다.

7-5. 객체 지향 프로그래밍의 특징

➢ 객체 지향 프로그램의 특징은 캡슐화, 상속, 다형성이다. 이러한 특징들은 자바 언어를 학습하면서 자연스럽게 알게 되는데, 여기서는 간단히 개념만 살펴 보도록 합니다.

캡슐화

캡슐화(Encapsulation)란 객체의 데이터(필드), 동작(메서드)을 하나로 묶고 실제 구현 내용을 외부에 감추는 것을 말한다. 외부 객체는 객체 내부의 구조를 알지 못하며 객체가 노출해서 제공하는 필드와 메서드만 이용할 수 있다.



필드와 메서드를 캡슐화하여 보호하는 이유는 외부의 잘못된 사용으로 객체가 손상되지 않도록 하는 데 있다. 자바는 캡슐화된 멤버를 노출시킬 것인지 숨길 것인지를 결정하기 위해 접근 제한자를 사용합니다.

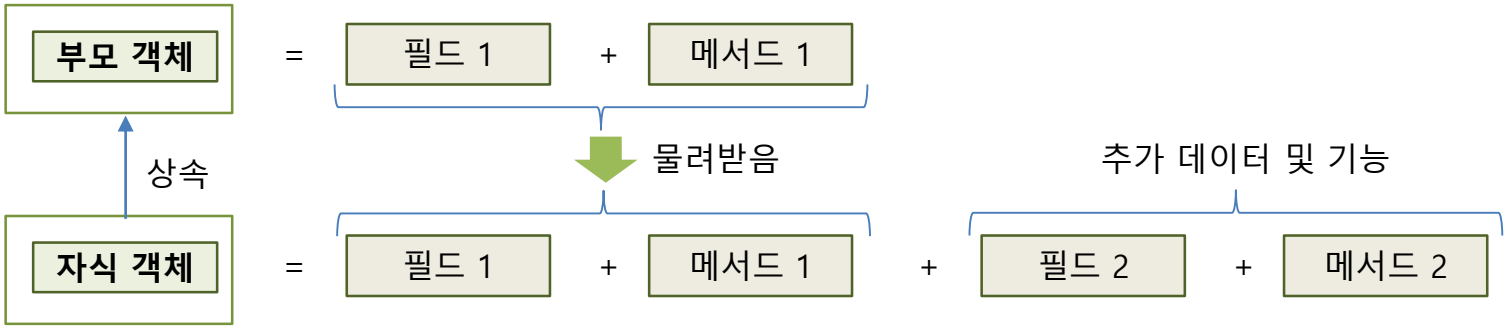
상속

객체 지향 프로그래밍에서는 부모 역할의 상위 객체와 자식 역할의 하위 객체가 있다. 부모 객체는 자기가 가지고 있는 필드와 메서드를 자식 객체에게 물려주어 자식 객체가 사용할 수 있도록 하는 데 이것을 상속(Inheritance)입니다. 상속을 하는 이유는 다음과 같습니다.

7-5. 객체 지향 프로그래밍의 특징

객체 지향 프로그래밍의 특징

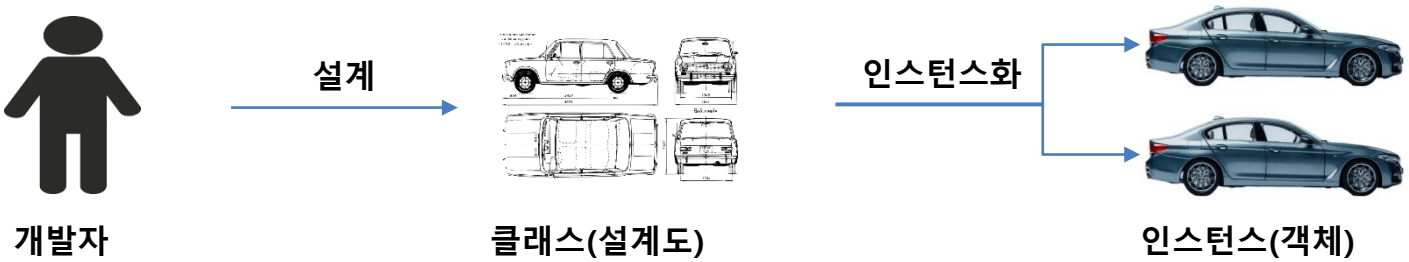
- 코드의 재사용성을 높여준다: 잘 개발된 부모 객체의 필드와 메서드를 자식 객체가 그대로 사용할 수 있어 중복 코딩을 방지
- 유지보수 시간을 최소화 시켜준다: 부모 객체의 필드와 메서드를 수정하면 모든 자식 객체들은 수정된 필드와 메서드를 사용할 수 있다.



다형성

다형성(Polymorphism)이란 사용 방법은 동일하지만 실행 결과가 다양하게 나오는 성질을 말합니다. 자동차 부품을 교환하면 성능이 다르게 나오듯이 프로그램을 구성하는 객체(부품)을 바꾸면 프로그램 실행 성능이 다르게 나올 수 있습니다. 다형성을 구현하기 위해서는 자동 타입 변환과 재정의 기술이 필요한데 이러한 기술들은 상속과 인터페이스 구현을 통해 얻어집니다. 상속과 인터페이스는 뒤에서 해당 내용을 다룰 때 설명하도록 하겠습니다.

7-6. 객체와 클래스

객체를 생성할 때에는 설계도가 필요하다. 현실 세계에서 자동차를 생성하려면 자동차의 설계도가 필요하듯이, 객체 지향 프로그래밍에서도 객체를 생성하려면 설계도에 해당하는 클래스(Class)가 필요하다.

클래스로부터 생성된 객체를 해당 클래스의 인스턴스(Instance)라고 부르고, 클래스로부터 객체를 만드는 과정을 인스턴스화라고 한다. 동일한 설계도로 여러 대의 자동차를 만들듯이 동일한 클래스로부터 여러 개의 인스턴스를 만들 수 있습니다.